

Uniwersytet Jagielloński w Krakowie
Wydział Fizyki, Astronomii i Informatyki Stosowanej

Michał Ojczenasz

Nr albumu: 1179518

Przedstawianie czterowymiarowych danych na płaszczyźnie

Praca licencjacka
na kierunku Fizyka dla firm

Praca wykonana pod kierunkiem
dr hab. Jakuba Prauzner-Bechcickiego, prof. UJ
Zakład Fizyki Nanostruktur i Nanotechnologii

Kraków 2023

Oświadczenie autora pracy

Świadom odpowiedzialności prawnej oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Oświadczam również, że przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w wyższej uczelni.

.....
Kraków, dnia

.....
Podpis autora pracy

Oświadczenie kierującego pracą

Potwierdzam, że niniejsza praca została przygotowana pod moim kierunkiem i kwalifikuje się do przedstawienia jej w postępowaniu o nadanie tytułu zawodowego.

.....
Kraków, dnia

.....
Podpis kierującego pracą

Spis treści

1	Wstęp teoretyczny	3
1	Oddziaływanie światła z materią	3
2	Wielokrotna jonizacja	4
3	Narzędzia	7
2	Redukcja danych trójwymiarowych	9
1	Dane testowe	9
2	Rzut na okrąg	10
3	Rzut na kwadrat	10
4	Dane zredukowane	12
3	Projekcja oktaedryczna	13
1	Dane testowe	14
2	Rzutowanie na sferę	14
3	Rzutowanie na oktaedr	14
4	Wizualizacja danych na płaszczyźnie	15
5	Optymalizacja	18
6	Inny przykład	19
4	Analiza danych z obliczeń numerycznych	21
5	Podsumowanie	25
A	Skrypt	27

Streszczenie

Praca jest poświęcona zagadnieniu wizualizacji danych wielowymiarowych na płaszczyźnie. Wymiernym efektem tej pracy jest przygotowany skrypt pozwalający na przeprowadzenie projekcji oktaedrycznej danych czterowymiarowych. Ponadto, skrypt pozwala na prezentację danych w postaci wykresów trójkątnych, a tym samym na ich redukcję. Wspomniana redukcja danych ma charakter bardzo ogólny, stąd też na potrzeby pracy zostały wykorzystane do analizy dane pochodzące z modelowania numerycznego zjawiska wielokrotnej jonizacji atomów w silnych polach laserowych. Pierwsza część pracy przybliży elementarne wiadomości na temat zjawiska jonizacji w ogóle, by następnie wprowadzić czytelnika w zagadnienie jonizacji w silnych polach laserowych. Problem redukcji zostaje przedstawiony na prostszym zagadnieniu przejścia z trzech do dwóch wymiarów oraz w docelowym przypadku z czterech do trzech. Rozwiązanie zostaje zaaplikowane w języku Python z wykorzystaniem standardowych bibliotek oraz jednej specjalistycznej.

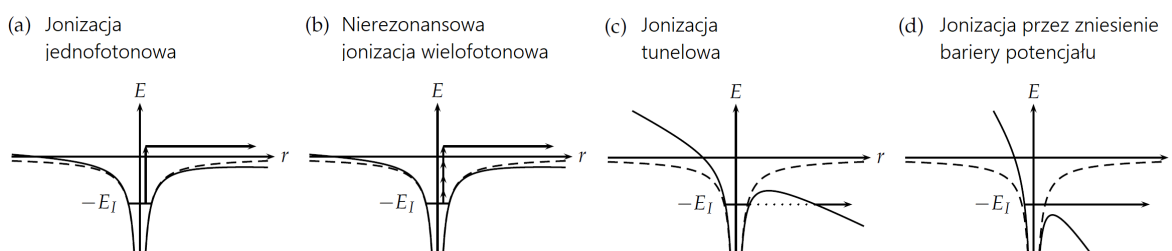
Rozdział 1

Wstęp teoretyczny

1 Oddziaływanie światła z materią

Światło może oddziaływać z materią na różne sposoby i prowadzić do wielu procesów. Pod wpływem promieni świetlnych materiały mogą się nagrzewać, czyli cząsteczki z których się składają zwiększają swoją energię kinetyczną. Dla światła materiały mogą być nieprzezroczyste i odbijające. Mogą być też przezroczyste i przepuszczać światło. W wyidealizowanym przypadku wcale go nie absorbują, a ewentualnie mogą zmienić kierunek jego propagacji. Światło może też spowodować zjawisko zwane jonizacją. Zjawisko jonizacji pod wpływem światła, polega na oddzieleniu elektronu bądź elektronów od reszty atomu w skutek absorpcji wystarczającej ilości energii (fotonów).

Gdy mówimy o jonizacji atomu, najczęściej mamy na myśli efekt foto-elektryczny, jednak istnieje wiele sposobów na jonizację atomu za pomocą interakcji ze światłem. Schemat kilku z takich zjawisk widnieje na Rys. 1.1. Poniżej krótki opis każdego z nich [1].



Rysunek 1.1: Oddziaływania światła z materią, prowadzące do jonizacji. Oś E oznacza energię, zaś r odległość od jądra atomu na jaką może się oddalić elektron w zależności od energii [2].

- (a) Efekt fotoelektryczny – Elektron znajdujący na powłoce elektronowej w atomie jest z nim związany określoną energią. Energia ta, zwana energią wiązania, jest zależną od potencjału jądra atomowego oraz powłoki na której się znajduje elektron. Jeśli oświetlimy atom światłem, którego fotony mają energię E_f większą niż energia wiązania E_I , to może on zaabsorbować foton i zyskać energię potrzebną do uwolnienia elektronu (patrz Rys. 1.1(a)):

$$E_f > E_I. \quad (1.1)$$

Energia fotonu E_f jest ściśle powiązana z częstotliwością ν i długością fali λ źródła światła. Z pomocą stałej Plancka h oraz prędkości światła w próżni c zależności te przedstawia się następująco:

$$E_f = h\nu = h\frac{c}{\lambda}. \quad (1.2)$$

- (b) Jonizacja wielofotonowa – Atom absorbuje kilka fotonów, aby uzyskać energię potrzebną do ucieczki. Takie zjawisko wymaga dostępności dużej ilości fotonów, czyli dużego natężenia światła. Suma energii fotonów musi być większa od energii wiązania:

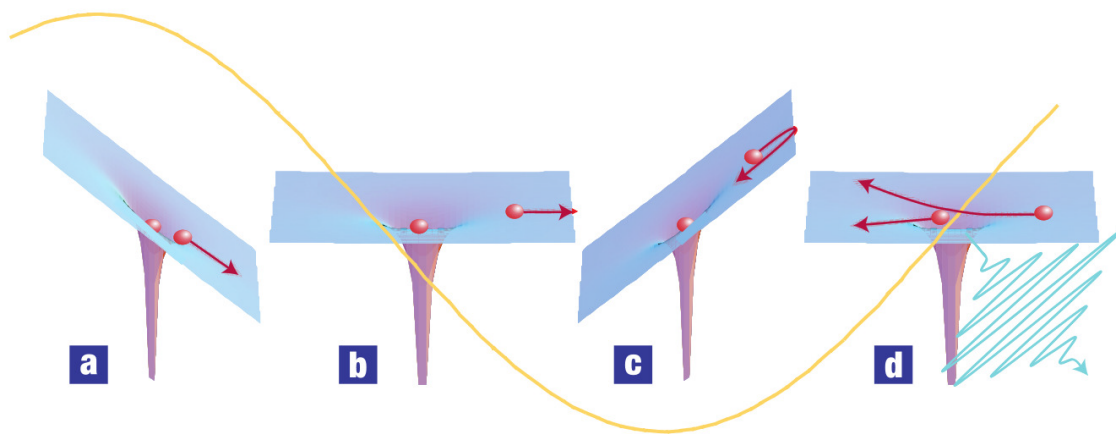
$$\sum_{i=1}^N h\nu_i > E_I. \quad (1.3)$$

Warto zaznaczyć, że to zjawisko da się opisać jako przekazanie energii fotonów, na wzbudzenie elektronu powyżej energii wiązania, nie biorąc pod uwagę zmiany potencjału atomu przez intensywne pole (patrz Rys. 1.1(b)).

- (c) Jonizacja tunelowa – Światło o wysokim natężeniu zmienia potencjał, w którym uwięziony jest elektron, przez co bariera potencjału staje na tyle wąska, że elektron jest w stanie ją przekroczyć w zjawisku zwanym tunelowaniem kwantowym (patrz Rys. 1.1(c)).
- (d) Jonizacja ponad barierą (ang. *over-the-barrier ionization*) – Światło tak bardzo zmienia potencjał wiążący elektron, że bariera potencjału przestaje go ograniczać i jest on w stanie opuścić atom (patrz Rys. 1.1(d)).

2 Wielokrotna jonizacja

W atomach wieloelektronowych może dochodzić do wielokrotnej jonizacji, jeśli tylko zostanie dostarczona wystarczająco duża energia. Zwykle gdy wspomina się o wielokrotnej jonizacji, traktuje się ten proces jako sekwencyjny. Sekwencyjny, czyli taki w którym elektrony są uwalniane pojedynczo, jeden za drugim. Jednak eksperymenty z lat 80 ubiegłego wieku wykazały, że gdy atomy oddziałują z krótkimi silnymi impulsami laserowymi spolaryzowanymi liniowo o częstościach nierezonansowych, to rejestrowana ilość podwójnych jonów jest o rzędy wielkości większa niż przewidywana na podstawie modelu sekwencyjnego [3, 4]. Taka rozbieżność sugeruje, że istnieje inny proces, który ma swój ogromny udział w powstawaniu podwójnych jonów. Zjawisko, które odpowiada za powstawanie znacznie większej ilości jonów wielokrotnych zostało nazwane niesekwencyjną wielokrotną jonizacją (z ang. *non-sequential multiple ionization*). Obecnie akceptowalny model tego zjawiska jest znany pod nazwą: model prostego człowieka (ang. *simple man model*) [5]. Model ten zostanie opisany poniżej, a jego schemat widnieje na Rys. 1.2.

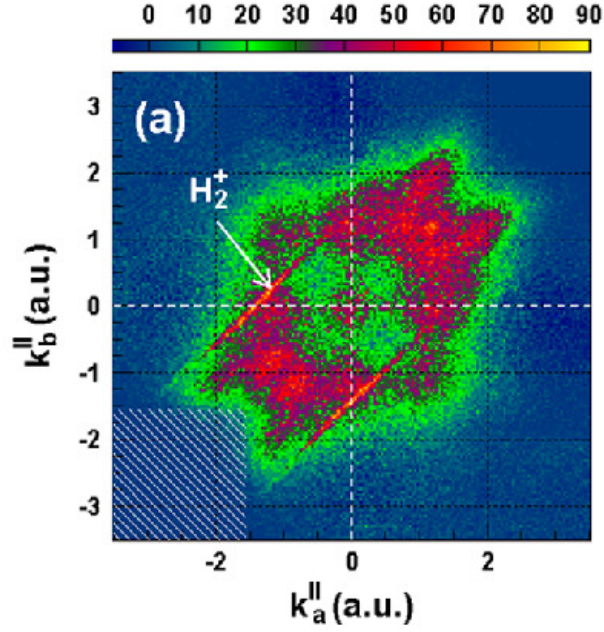


Rysunek 1.2: Ilustracja trójstopniowego modelu niesekwencyjnej jonizacji w silnych polach laserowych [5]. Schematyczne przedstawienie zmian potencjału wiążącego elektron w atomie spowodowanych oddziaływaniem ze zmiennym polem elektrycznym fali elektro-magnetycznej: obniżenie bariery potencjału i tunelowanie (panel (a)), ucieczka i zawrótce elektronu (panele (b) oraz (c)), wtórne rozpraszanie zakończone jonizacją lub emisją wysokich harmonicznych (panel (d)) [6].

Uwolniony elektron znajdujący się w pobliżu atomu zostaje przyspieszony przez pole elektryczne (światło) w kierunku polaryzacji światła (Rys. 1.2(a)). Po pewnym czasie pole elektryczne zmienia zwrot i elektron zaczyna być wyhamowywany (Rys. 1.2(b)), a następnie przyspieszany z powrotem w kierunku macierzystego jonu (Rys. 1.2(c)). W momencie gdy elektron trafia z powrotem na jon może się wydarzyć kilka scenariuszy (Rys. 1.2(d)):

- Nadlatujący elektron w zderzeniu niesprężystym, przekazuje część energii elektronowi w macierzystym jonie i razem opuszczają atom, aby to nastąpiło energia powracającego elektronu musi być większa niż energia jonizacji związanego elektronu.
- Powracający elektron przelatuje przez macierzysty jon. Po drodze wzbudza związanego elektron w zderzeniu niesprężystym. Na skutek czego tak wzbudzony jon może wyemitować elektron w późniejszym czasie, niejako niezależnie od rozpraszanego elektronu.
- Wtórnie rozpraszany elektron zostaje wychwycony przez atom, a nadmiar energii atom wypromieniowuje w postaci fali elektromagnetycznej (tzw. generacja wysokich harmonicznych).

Najprostszym układem w którym możliwe jest zaobserwowanie eksperymentalne opisanego zjawiska jest hel. Jest to wygodne ze względu na jego neutralność chemiczną oraz posiadanie dwóch elektronów. Z tego powodu szereg prac eksperymentalnych było poświęconych badaniu niesekwencyjnej jonizacji helu. W typowych pomiarach mierzy się pęd jonu podwójnego p_j i jednego elektronu w koincydencji p_e . Pomiaru dokonuje się wzdłuż kierunku wyznaczonego przez polaryzację światła. Pęd drugiego elektronu oblicza się z zasady zachowania pędu, przy założeniu, że fotony mają znikomy wkład do pędu. Hel jest wygodny do analizy teoretycznej, która sprowadza się przede wszystkim do numerycznych obliczeń w ramach



Rysunek 1.3: Rozkład pędów elektronów przy podwójnej niesekwencyjnej jonizacji helu za pomocą liniowo spolaryzowanego lasera, o długości fali 800 nm i czasie trwania 40 fs, o maksymalnej intensywności $4.5 \cdot 10^{14}$ W/cm² [7].

mechaniki klasycznej lub kwantowej. Przy obliczeniach realizowanych w ramach mechaniki klasycznej możliwe jest obliczenie wszystkich składowych położeń i pędów zarówno dla elektronów jak i jonu. Z tego powodu już na tym etapie pojawiają się trudności z wizualizacją danych. W celu ominięcia tego problemu, do analizy wykorzystuje się jedynie składowe pędów równoległe do osi liniowej polaryzacji lasera. Przykładowy obraz rozkładu pędów elektronów otrzymany z analizy danych eksperymentalnych przedstawiony jest na Rys. 1.3, kolor oznacza częstość takich zdarzeń. Nawet z pobieżnej analizy przedstawionego rozkładu widać, że uciekające elektrony opuszczają atom w skorelowany sposób.

Jeżeli do procesu opisanego powyżej zostanie wykorzystany atom o większej ilości elektronów niż 2, to w momencie wtórnego rozpraszania powracającego elektronu (Rys. 1.2(d)), w zderzeniu niesprężystym, może zostać wybity nie jeden, a dwa lub więcej dodatkowych elektronów.

Eksperymenty zmierzające do pomiarów pędów wszystkich uwolnionych elektronów w przypadku jonizacji wyższych niż podwójna są bardzo trudne. Według mojej wiedzy, wciąż czekają na realizację. Opis teoretyczny potrójnej jonizacji sam w sobie jest bardzo trudnym zagadnieniem. Przewagę zyskują tutaj metody mechaniki klasycznej, które pozwalają na uzyskanie w obliczeniach pełnych trajektorii w przestrzeni fazowej dla wszystkich jej składników (jonu oraz trzech elektronów [8]). Natomiast obliczenia kwantowe są tak wymagające pod względem potrzebnej do nich mocy obliczeniowej (między innymi możliwości operowania ogromną pamięcią podręczną), że aby były możliwe do realizacji konieczne jest stosowanie modeli uproszczonych np. o zmniejszonej liczbie wymiarów [9]. Za punkt wyjścia weźmiemy obliczenia w ramach mechaniki klasycznej. Tu jednak powstaje nowy problem, dobrze by było zaprezentować rozkłady pędów dla 3 elektronów analogicznie do Rys. 1.3. Nawet gdy

wyberzemy składowe pędów wzdłuż osi polaryzacji, to chcielibyśmy pokazać liczbę zliczeń w funkcji z trzema argumentami - składowe pędów trzech elektronów. W przestrzeni trójwymiarowej najtrudniejszym elementem do przedstawienia jest wartość funkcji w danym punkcie. Przy małej ilości pojedynczych punktów można próbować zmieniać intensywność rozmiar czy kolor, ale przy dużej ilości danych staje się to bardzo nieczytelne. Z tego względu konieczne jest jakieś rzutowanie, które zachowa wybrane istotne informacje. Jedną z możliwości takiego rzutu, jest projekcja oktaedryczna. Zachowuje ona informacje o stosunkach pędów pomiędzy składowymi punktów.

3 Narzędzia

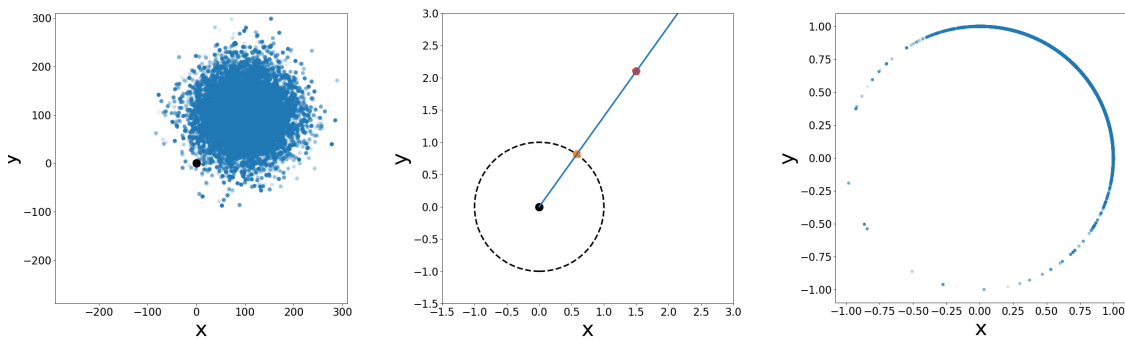
Skrypt do analizy i graficznej prezentacji danych został napisany w języku Python. W swojej pracy wykorzystałem środowisko JupyterLab, lecz przedstawione rozwiązania są od niego niezależne. Natomiast konieczne jest zastosowanie standardowych bibliotek Pythona: matplotlib, numpy oraz pandas, a także jednej specjalistycznej biblioteki umożliwiającej tworzenie trójkątnych wykresów: mpltern [10].

Rozdział 2

Redukcja danych trójwymiarowych

W pierwszej kolejności przedstawiona zostanie redukcja danych trójwymiarowych do dwuwymiarowych. Bez trudu można przedstawić dane przed redukcją, by następnie porównać je z reprezentacją zredukowaną. Daje to możliwość wyrobienia sobie intuicji co do redukcji danych w problemie o wyższej wymiarowości. Takie przekształcenie może odpowiadać przypadkowi z analizą dwóch pędów elektronów, wtedy wynik przekształcenia będzie funkcją rozłożenia całkowitego pędu pomiędzy nimi. Punktem wyjścia jest funkcja $z = f(x, y)$, która po przekształceniu zostanie zredukowana do czterech funkcji $z = g_i(\hat{x})$. Warto zauważyć, że wartości funkcji nie ulegną zmianie. Proces redukcji danych składa się z projekcji na okrąg, projekcji na kwadrat, a na koniec z utworzenia 4 funkcji z punktów znajdujących się na każdym z boków kwadratu. Każdy etap procesu, zostanie zilustrowany i opisany w osobnych podrozdziałach.

1 Dane testowe



Rysunek 2.1: Lewy panel: rozkład losowo wygenerowanych danych na płaszczyźnie. Środkowy panel: schemat rzutowania punktu. Prawy panel: obraz danych rzutowanych na okrąg.

Punktem wyjścia jest funkcja $z = f(x, y)$. Na potrzeby wizualizacji zostały wygenerowane losowe dane. Argumenty funkcji w postaci wartości x i y zostały wygenerowane z rozkładu Gaussa o parametrach $\mu = 100$ oraz $\sigma = 10$. Wartości funkcji zostały wygenerowane z

rozkładu jednorodnego na przedziale $\langle 0, 1 \rangle$. Na wykresach zostały one uwzględnione w postaci stopnia przeźroczystości. Wartość 0, daje nam całkowicie niewidoczny punkt, a wartość 1 w pełnym kolorze. Każda para argumentów funkcji $f(x, y)$ w dalszej części tekstu będzie oznaczana jako punkt $P = (x, y)$. Obraz aktualnie opisywanego kroku przekształcenia będzie oznaczany jako $P' = (x', y')$. Wartość funkcji w punkcie P' będzie taka sama jak punkcie P . Wygenerowane dane przedstawiam na Rys. 2.1. Czarnym punktem został zaznaczony środek układu współrzędnych.

2 Rzut na okrąg

Wstępnym elementem przekształcenia jest umieszczenie każdego punktu z wyjściowego rozkładu na okręgu. Dla każdego z punktów P (czerwony punkt Rys. 2.1 środkowy panel) została przeprowadzona prosta przez ten punkt oraz początek układu współrzędnych. Następnie w punkcie przecięcia znalezionej prostej z okręgiem o wybranym promieniu, umieszczamy punkt P' (pomarańczowy punkt Rys. 2.1 środkowy panel). Dla uproszczenia obliczeń numerycznych w skrypcie został zastosowany inny sposób. Celem jest umieszczenie obrazu wszystkich punktów na okręgu, czyli w odległości r od środka układu. Zatem wystarczy sprawdzić aktualną odległość $r_i = \sqrt{x_i^2 + y_i^2}$ każdego z punktów, a następnie każdą składową położenia podzielić przez odpowiednio przeskalowaną odległość r za pomocą stałej. Jeśli chcemy aby promień okręgu był równy jeden, wystarczy posłużyć się wzorem:

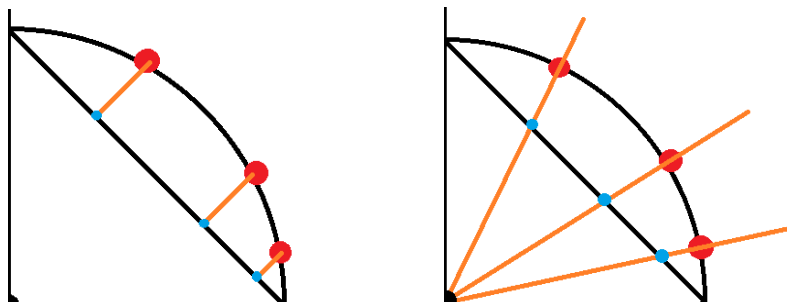
$$\begin{cases} r_i &= \sqrt{x_i^2 + y_i^2} \\ y'_i &= \frac{y_i}{r_i} \\ x'_i &= \frac{x_i}{r_i} \end{cases} . \quad (2.1)$$

Obraz danych po przeprowadzeniu powyższego procesu widnieje na prawym panelu Rys. 2.1.

3 Rzut na kwadrat

Następnym etapem jest rzut z okręgu na kwadrat do niego wpisany. Przekątne kwadratu są równoległe do osi współrzędnych, a każdy z boków znajduje się w jednej ze ćwiartek układu. Poniżej krótki opis dwóch wybranych realizacji tego rzutu.

- Pierwszy z nich to znalezienie prostej prostopadłej do boku kwadratu i przechodzącej przez dany punkt P . Punkt P' jest umieszczany w punkcie przecięcia znalezionej prostej i boku kwadratu, którego dotyczy (Rys. 2.2, lewy panel).
- Drugim sposobem, jest znalezienie prostej przechodzącej przez środek układu współrzędnych i punkt P . Punktu P' zostaje umieszczony w miejsce przecięcia znalezionej prostej z prostą opisującą dany bok kwadratu (Rys. 2.2 prawy panel).

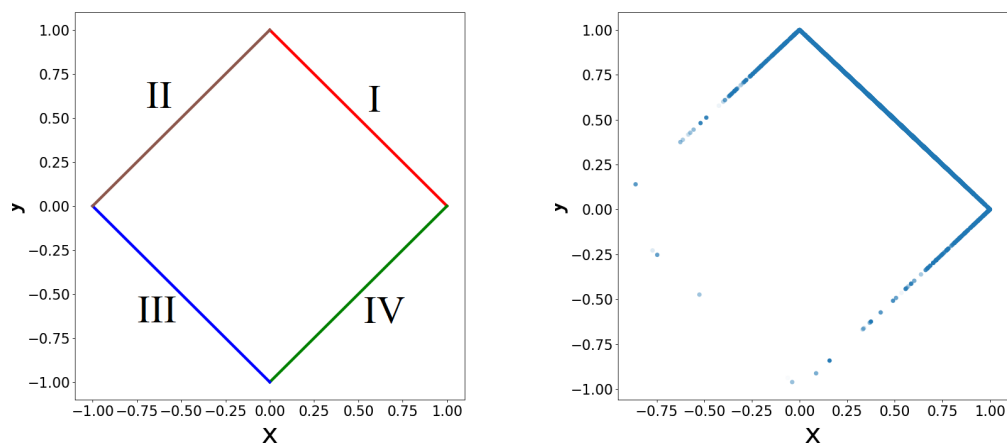


Rysunek 2.2: Po lewej: projekcja wzdłuż prostych prostopadłych do boku. Po prawej: projekcja wzdłuż prostych przeprowadzonych każdy z punktów i środek układu. Czerwone kropki reprezentują punkty P_i , a niebieskie P'_i .

W każdym z tych przekształceń przy obliczaniu położenia obrazu punktu, należy brać pod uwagę ćwiartkę, w której się dany punkt znajduje. Równania prostych pokrywających boki kwadratu dla poszczególnych ćwiartek:

$$I : y = -x + 1, \quad II : y = x + 1, \quad III : y = -x - 1, \quad IV : y = x - 1. \quad (2.2)$$

Ze względu na charakter docelowo przekształcanych danych, w których interesują nas zachowane stosunki wartości x i y , została zastosowana druga metoda, czyli przeprowadzenie prostej prowadzącej przez środek układu i rzutowany punkt. Można zauważyć, że ten krok pokrywa się z poprzednim (rzutowaniem na okrąg). Z tego względu w tym rzucie można pominąć krok z okręgiem, a punkty przerzucać od razu na boki kwadratu. Wszystkie proste przechodzące przez punkt $(0, 0)$ mają wyraz wolny $b = 0$. Dzięki temu łatwo znaleźć współczynnik kierunkowy takiej prostej, Rów. (2.3). Równanie (2.4) opisuje schemat tworzenia obrazów punktów P' w zależności od ćwiartki układu.



Rysunek 2.3: Po lewej: boki kwadratu w poszczególnych ćwiartkach. Po prawej: obraz danych po rzucie na kwadrat.

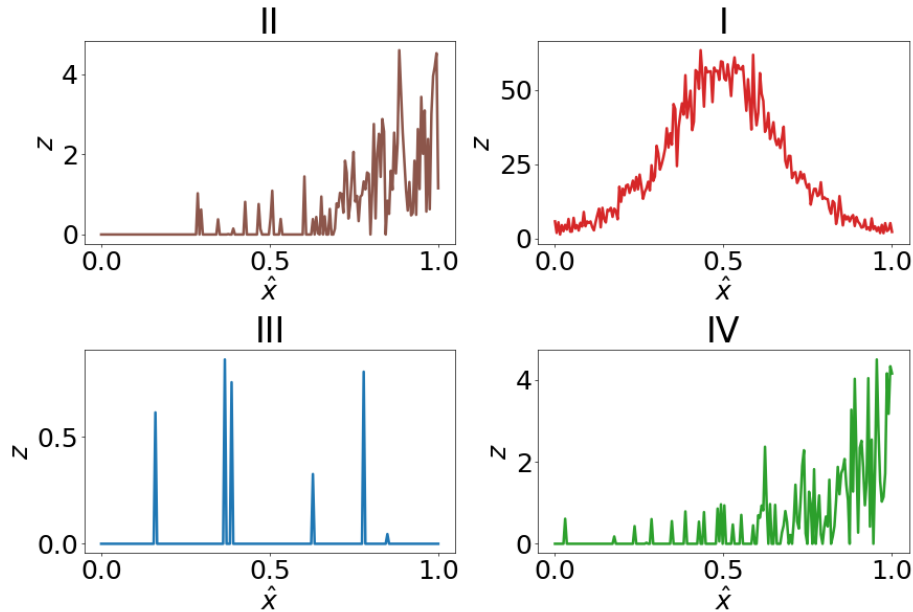
$$y_i = ax_i \implies a = \frac{y_i}{x_i}. \quad (2.3)$$

$$I \begin{cases} x'_i = \frac{1}{a+1} \\ y'_i = \frac{a}{a+1} \end{cases}, \quad II \begin{cases} x'_i = \frac{1}{a-1} \\ y'_i = \frac{a}{a-1} \end{cases}, \quad III \begin{cases} x'_i = \frac{-1}{a+1} \\ y'_i = \frac{-a}{a+1} \end{cases}, \quad IV \begin{cases} x'_i = \frac{-1}{a-1} \\ y'_i = \frac{-a}{a-1} \end{cases}. \quad (2.4)$$

Wynik przekształcenia danych testowych według przepisu opisanego w tym kroku przedstawia Rys. 2.3.

4 Dane zredukowane

Ostatnim etapem jest rozłożenie kwadratu na którym aktualnie znajduje się już obraz danych, na osobne boki. Uzyskane boki są już tylko jednowymiarowe, dzięki temu dla każdego z boków można zdefiniować nową funkcję $g(\hat{x}) = z$. Wartość $\hat{x}$ reprezentuje odległość od jednego z wierzchołków kwadratu należącego do danego boku. Jeśli projekcja zostanie przeprowadzona na pędach, to warto znormalizować argumenty funkcji $\hat{x}$, tak by $\hat{x} \in \langle 0, 1 \rangle$ (przenosząc równocześnie wartości funkcji). Wtedy funkcja w bezpośredni sposób mówi o częstotliwości zjawisk, w zależności od stosunku pędów pomiędzy elektronami. Nowe funkcje da się w prosty sposób zaprezentować na płaszczyźnie (Rys. 2.4).

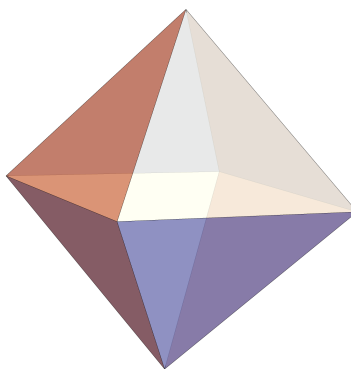


Rysunek 2.4: Rozkłady w ćwiartkach. W pierwszej ćwiartce intensywność punktów zlokalizowana jest głównie na środku ściany. W drugiej i czwartej ćwiartce rozkłady są mniej-więcej symetryczne ze względu na zamianę składowych x i y . Oba mają tendencje do lokalizowania się przy krawędzi z pierwszą ćwiartką. W trzeciej ćwiartce znajdują się tylko pojedyncze punkty, które zostały wygenerowane w "ogonach" dwuwymiarowej funkcji Gaussa.

Rozdział 3

Projekcja oktaedryczna

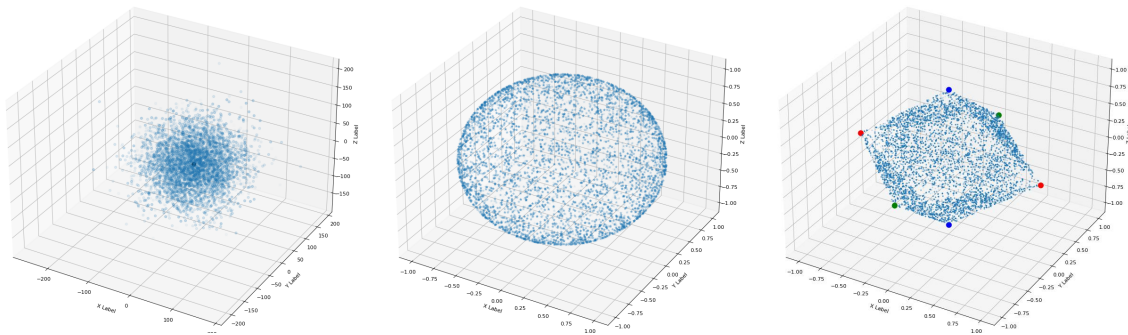
Mając już bardziej przyjazne podstawy, łatwiejsza do wyobrażenia będzie projekcja oktaedryczna (przy zwiększonej ilości wymiarów). Oktaedr (ośmiościan foremny) to bryła, wyglądająca jak dwie piramidy sklejone spodami, których boki są zbudowane z trójkątów równobocznych (Rys. 3.1). Projekcja będzie przeprowadzona w pełnej analogii do poprzedniej. Różnice będą polegać na początkowym rzutowaniu na sferę zamiast na okrąg. Następnie na oktaedr, zamiast na kwadrat. Odzyskanie informacji z boków będzie miało nieco inną formę, ponieważ wynikiem końcowym będzie wykres trójkątny. Punktem wyjścia jest funkcja $v = h(x, y, z)$, która po przekształceniu zostanie zredukowana do ośmiu funkcji $v = h(d_{xy}, d_{yz}, d_{zx})$. Wartości d_{xy}, d_{yz}, d_{zx} posiadają taką właściwość, że $d_{xy} + d_{yz} + d_{zx} = 1$. Każdy etap procesu, zostanie zilustrowany i opisany w osobnych podrozdziałach. Punkty w przekształceniu definiujemy jako $K = (x, y, z)$, a obraz tych punktów w danym kroku jako $K' = (x', y', z')$. Przekształcenie jest prezentowane na sztucznie wygenerowanych danych. W taki sposób by łatwo było je porównać na każdym etapie przekształcenia.



Rysunek 3.1: Ośmiościan foremny, zwany oktaedrem. Narysowany z pomocą programu Mathematica [11].

1 Dane testowe

Argumenty funkcji zostały wygenerowane z trzech rozkładów Gaussa: $x, y, z \in N(\mu = 0, \sigma = 50)$. Wartość funkcji w punktach $v = h(x, y, z)$ została wygenerowana z rozkładu jednorodnego $v \in \langle 0, 1 \rangle$.



Rysunek 3.2: Na lewym panelu: wygenerowane dane. Czarną kropką zaznaczony punkt $(0, 0, 0)$. Na środku: obraz punktów na sferze. Po prawej: obraz punktów rzutowanych na oktaedr.

2 Rzutowanie na sferę

Gdy każdą współrzędną rzutowanego punktu K podzielimy przez całkowitą odległość punktu od początku układu współrzędnych, to punkt zostanie przeniesiony na sferę o środku w punkcie $(0, 0, 0)$ i promieniu 1 (Rys. 3.2 środkowy panel).

$$\begin{cases} r_i = \sqrt{x_i^2 + y_i^2 + z_i^2} \\ x'_i = \frac{x_i}{r_i} \\ y'_i = \frac{y_i}{r_i} \\ z'_i = \frac{z_i}{r_i} \end{cases} \quad (3.1)$$

3 Rzutowanie na oktaedr

Przy rzucie ze sfery na oktaedr, podobnie jak przy rzutowaniu danych z okręgu na boki kwadratu, również istnieją różne możliwości realizacji takiego rzutu. Jeden sposób to poprowadzenie prostych prostopadłych do danej płaszczyzny opisującej bok i przeniesienie danych wzdłuż tych prostych, na płaszczyznę. Drugi sposób polega na przeprowadzeniu prostych pomiędzy środkiem oktaedru, a każdym punktem i przeniesieniu każdego punktu wzdłuż takiej prostej na płaszczyznę. Został wykorzystany ten drugi sposób ze względu na to, że zachowuje on istotną informację o proporcjach pomiędzy składowymi położenia punktów. Warto dodać, że stosując ten sposób (podobnie jak w przypadku z mniejszą ilością wymiarów) można pominąć krok z rzutem na sferę. Krok ten został tutaj ujęty dla lepszego zrozumienia co się dzieje z danymi. Dla przypomnienia, równanie ogólne płaszczyzny w trzech wymiarach:

$$Ax + By + Cz + D = 0. \quad (3.2)$$

Równania (3.3) opisują 8 płaszczyzn pokrywających boki wybranego oktaedru, którego wierzchołki znajdują się na sferze o promieniu 1. Zapis „-” lub „+” oznaczają znak danej osi w kolejności XYZ.

$$\begin{aligned} (+ + +) : +x + y + z - 1 &= 0, & (+ + -) : +x + y - z - 1 &= 0, \\ (- + +) : -x + y + z - 1 &= 0, & (- + -) : -x + y - z - 1 &= 0, \\ (- - +) : -x - y + z - 1 &= 0, & (- - -) : -x - y - z - 1 &= 0, \\ (+ - +) : +x - y + z - 1 &= 0, & (+ - -) : +x - y - z - 1 &= 0. \end{aligned} \quad (3.3)$$

Aby przerzucić dane na boki oktaedru wystarczy wykorzystać równanie płaszczyzny na którą mają zostać rzutowane punkty. Dla przykładu zajmijmy się punktami znajdującymi się w oktancie (+ + +) wiemy, że aby znalazły się na boku w tym oktancie, muszą spełnić równanie płaszczyzny z Rów. (3.3). Czyli suma współrzędnych x, y, z powinna wynosić 1. Dla aktualnego punktu, można wyobrazić sobie płaszczyznę równoległą do płaszczyzny opisującej boku o równaniu:

$$x_i + y_i + z_i = D_i. \quad (3.4)$$

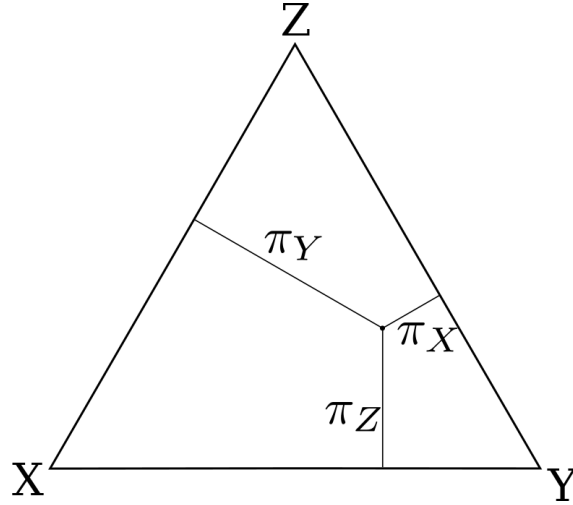
Aby punkty znalazły się na płaszczyźnie opisanej równaniem w Rów. (3.3), wystarczy podzielić obie strony Rów. (3.4) przez D_i .

$$\frac{x_i}{D_i} + \frac{y_i}{D_i} + \frac{z_i}{D_i} = 1. \quad (3.5)$$

Jeśli każda współrzędna zostanie podzielona przez sumę wszystkich składowych (sumę ze względu na równanie płaszczyzny), to punkt który opisują zostanie przeniesiony na płaszczyznę wzdłuż prostej przechodzącej przez środek układu oraz ten punkt. Tym sposobem oraz przy zastosowaniu odpowiedniego równania płaszczyzny w zależności od tego w jakim oktancie układu znajduje się punkt, został stworzony obraz punktów na oktaedrze (Rys. 3.2 prawy panel).

4 Wizualizacja danych na płaszczyźnie

Na tym etapie wszystkie punkty znajdują się na oktaedrze. By uzyskać obraz na płaszczyźnie, można zastosować na przykład macierz przekształceń z 3D na 2D. Jednak dla opisywanego przypadku wygodne będzie skorzystanie z wykresu trójkątnego. Do tego celu zostanie wykorzystana własność trójkąta równobocznego. Dla dowolnego punktu znajdującego się wewnątrz trójkąta równobocznego suma odległości od każdego z boków jest zawsze równa jednej wysokości tego trójkąta. Dzięki temu położenie punktu na wykresie trójskładnikowym można określić poprzez odległość punktu od każdego z boków trójkąta jeszcze w 3D (Rys. 3.3).



Rysunek 3.3: Położenie punktu na wykresie trójkątnym. Składowe sumują się do jedności. π_X oznacza odległość punktu od krawędzi YZ . Ta odległość daje informacje o proporcji wartości X w stosunku do dwóch pozostałych (Y i Z) [9].

Nowym zadaniem staje się obliczenie tej odległości. Bok trójkąta można opisać za pomocą prostej. Każda krawędź oktaedru, (czyli każdy bok trójkątnej ściany) znajduje się na płaszczyźnie, gdzie jedna ze współrzędnych jest równa 0. Przykładowo przeanalizuję zagadnienie dla pierwszego oktantu $(+++)$. Zauważmy, że jeśli w równaniu ogólnym płaszczyzny jedna ze składowych, przykładowo z jest równa 0, to równanie zamienia się w równanie prostej w dwóch wymiarach opisujące krawędź xy :

$$Ax + By + D = 0. \quad (3.6)$$

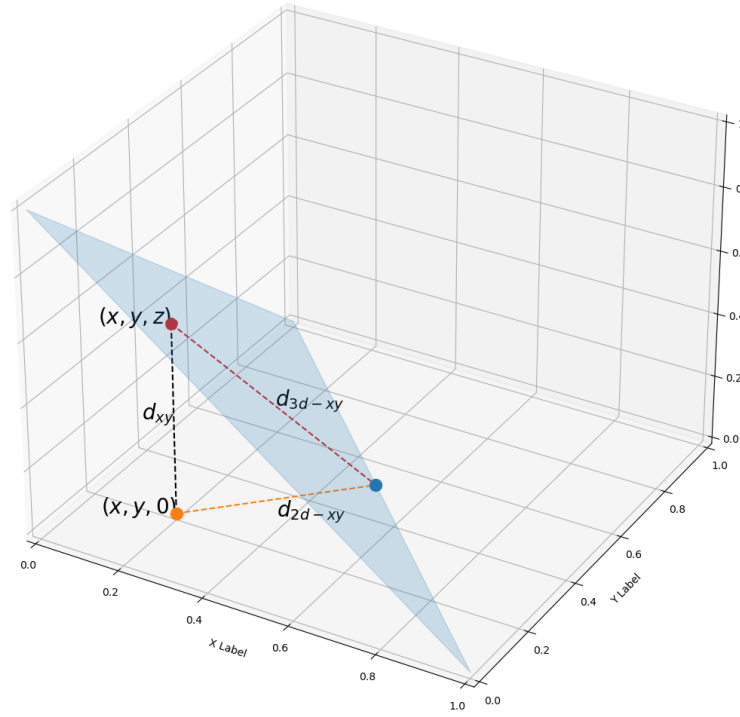
Obliczając odległości punktu $K = (x, y, z)$ od krawędzi znajdującej się na płaszczyźnie $x \times y$ można rzutować punkt na tę płaszczyznę (czyli tymczasowo zignorować wartość z). Dzięki temu problem zostanie sprowadzony do obliczenia odległości punktu od prostej w 2d. Oznaczmy tę odległość jako d_{2d-xy} (Rys. 3.4). Do policzenia odległości punktu od prostej zostanie wykorzystany wzór:

$$d_{2d-xy} = \frac{|Ax_i + By_i + D|}{\sqrt{A^2 + B^2}}. \quad (3.7)$$

Dodatkowo, dla omawianego oktaedru współczynniki A , B i C , dla wszystkich płaszczyzn spełniają zależność $A^2 = B^2 = C^2 = 1$, z tego względu równanie (3.7) można skrócić do postaci:

$$d_{2d-xy} = |Ax_i + By_i + D| \frac{1}{\sqrt{2}}. \quad (3.8)$$

Policzoną odległość d_{2d-xy} oraz pominiętą wcześniej składową z , użyjemy we wzorze z twierdzenia Pitagorasa, by policzyć odległość od prostej i punktu znajdującego się w trzech wymiarach, d_{3d-xy} :



Rysunek 3.4: Punkt K (na czerwono) wraz z obrazem jego rzutu na płaszczyznę $x \times y$ (na pomarańczowo).

$$d_{3d-xy} = \sqrt{d_{2d-xy}^2 + z_i^2}. \quad (3.9)$$

Poniżej równania dla wszystkich krawędzi w pierwszym oktancie. Kolejno krawędzie znajdujące się na płaszczyznach $x \times y$ Rów. (3.10), $y \times z$ Rów. (3.11), $z \times x$ Rów. (3.12).

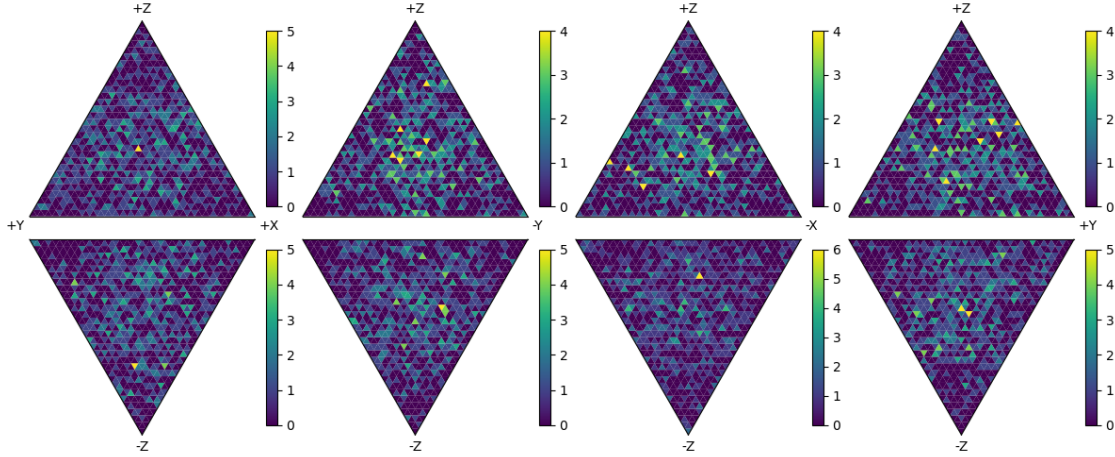
$$d_{xy} = \sqrt{\frac{(x_i + y_i - 1)^2}{2} + z_i^2}, \quad (3.10)$$

$$d_{yz} = \sqrt{\frac{(y_i + z_i - 1)^2}{2} + x_i^2}, \quad (3.11)$$

$$d_{zx} = \sqrt{\frac{(z_i + x_i - 1)^2}{2} + y_i^2}. \quad (3.12)$$

Aby móc skorzystać z wykresu trójkątnego należy jeszcze znormalizować dane, czyli wszystkie odległości powinny sumować się do 1:

$$\left\{ \begin{array}{lcl} suma & = & d_{xy} + d_{yz} + d_{zx} \\ d_{xy} & = & \frac{d_{xy}}{suma} \\ d_{yz} & = & \frac{d_{yz}}{suma} \\ d_{zx} & = & \frac{d_{zx}}{suma} \end{array} \right. . \quad (3.13)$$



Rysunek 3.5: Pocięta siatka oktaedru w postaci wykresów trójkątnych. Wszystkie ściany zostały ułożone tak, aby każda krawędź pasowała do najbliższego sąsiada. W lewym górnym rogu pierwszy oktant (+ + +).

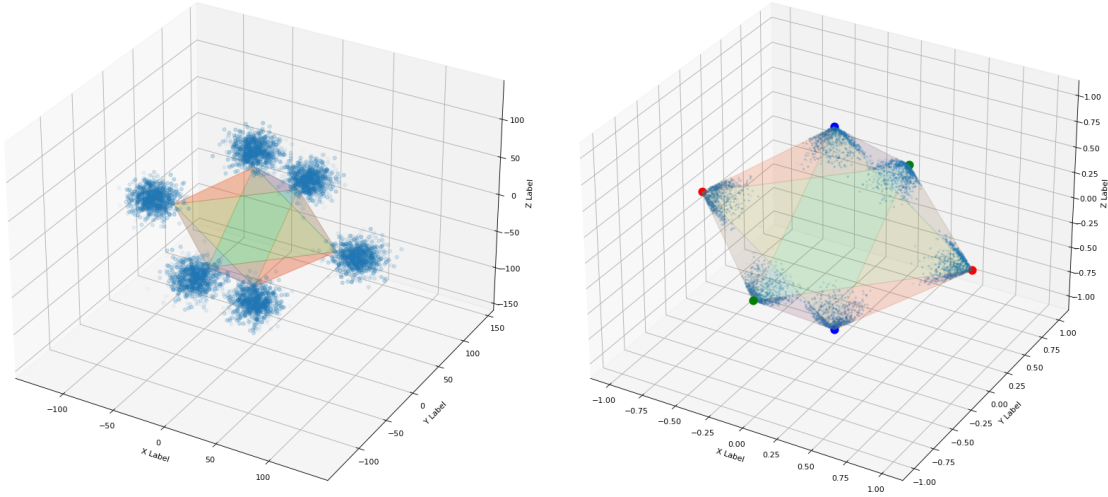
5 Optymalizacja

Po przyglądnięciu się aspektom obliczeniowym, właściwie każdą z pośrednich procedur można pominąć i przejść bezpośrednio do ostatniej. Okazuje się, że znak przy współczynnikach A , B i C w równaniu prostych, Rów. (3.8), zmienia się wraz ze znakiem składowej punktu, tak że po przemnożeniu zawsze dają wartość nieujemną. Tak samo zachowują się współczynniki w równaniu płaszczyzn. Korzystając z tego faktu można obłożyć w równaniu składowe wartością bezwzględną. Da nam to równanie na uniwersalne przekształcenie każdego punktu, niezależne w jakim oktancie się znajduje. W równaniu (3.4) wszystkie składowe zostały podzielone przez D_i . Jednak można dokonać obliczeń na dowolnie dużej płaszczyźnie równoległej do danego boku oktaedru. Można więc wybrać tą na której właśnie znajduje się punkt. Dzielenie wykonać dopiero na gotowej odległości punktu od krawędzi boku, zmniejszając to ilość działań. Aby wrzucić dane do wykresu trójkątnego najpierw należy je znormalizować. Po obliczeniu długości jaką posiada każda krawędź wybranego oktaedru $a = \sqrt{2}$, można łatwo obliczyć wysokość każdej ze ścian, jest to $\frac{\sqrt{6}}{2}$. Jeśli podzielimy każdą z odległości d_{xy} , d_{yz} , d_{zx} , przez $\frac{\sqrt{6}}{2}$ to otrzymamy wartości już znormalizowane. Po rozpisaniu i uproszczeniu, cały proces sprowadzi się do:

$$\left\{ \begin{array}{l} D_i = |x_i| + |y_i| + |z_i| \\ d_{xy} = \frac{2}{D_i\sqrt{6}} \sqrt{\frac{(|x_i|+|y_i|-D_i)^2}{2} + z_i^2} = \frac{|z_i|}{D_i} \\ d_{yz} = \frac{2}{D_i\sqrt{6}} \sqrt{\frac{(|y_i|+|z_i|-D_i)^2}{2} + x_i^2} = \frac{|x_i|}{D_i} \\ d_{zx} = \frac{2}{D_i\sqrt{6}} \sqrt{\frac{(|z_i|+|x_i|-D_i)^2}{2} + y_i^2} = \frac{|y_i|}{D_i} \end{array} \right. . \quad (3.14)$$

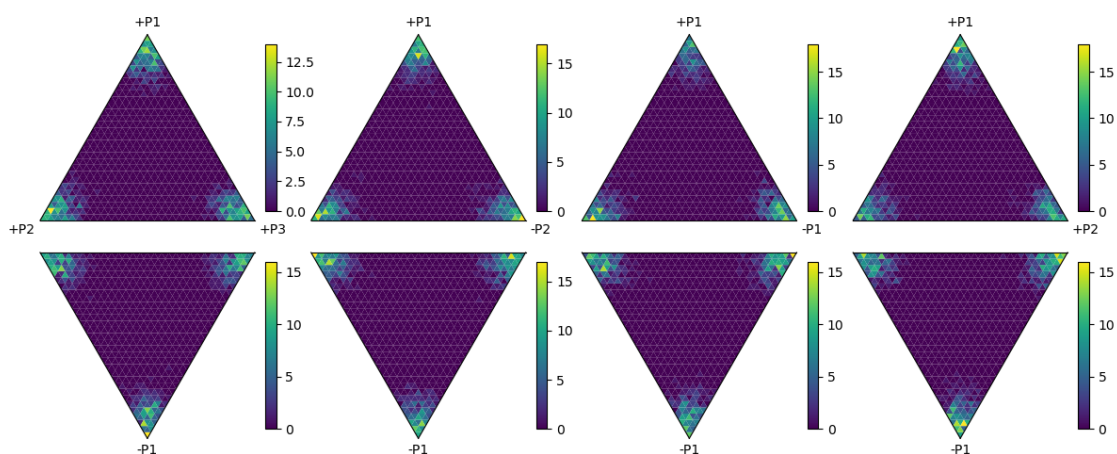
6 Inny przykład

Inne przykładowe dane, które powinny pomóc zbudować intuicję o działaniu projekcji, to przestrzenne rozkłady Gaussa o środkach w punktach gdzie jedna składowa jest równa 100 lub -100 , a pozostałe 0 (Rys. 6).



Rysunek 3.6: Na lewym panelu: wygenerowane dane, wraz z oktaedrem dającym pogląd na ułożenie danych w przestrzeni. Po prawej: obraz punktów rzutowanych na oktaedr.

Po rozłożeniu oktaedru na pojedyncze ściany (Rys. 3.7), obserwujemy rozkład punktów skupiony w rogach każdej ze ścian. Oznaczenie osi za pomocą liter x , y , z miało służyć, bardziej intuicyjnemu odnalezieniu się w przestrzeni i równaniach. Jednak narzędzie ma przetwarzać dane by ułatwić analizę pędów trzech różnych elektronów, dlatego od teraz nazwy osi zostały oznaczone jako $P1$, $P2$, $P3$.

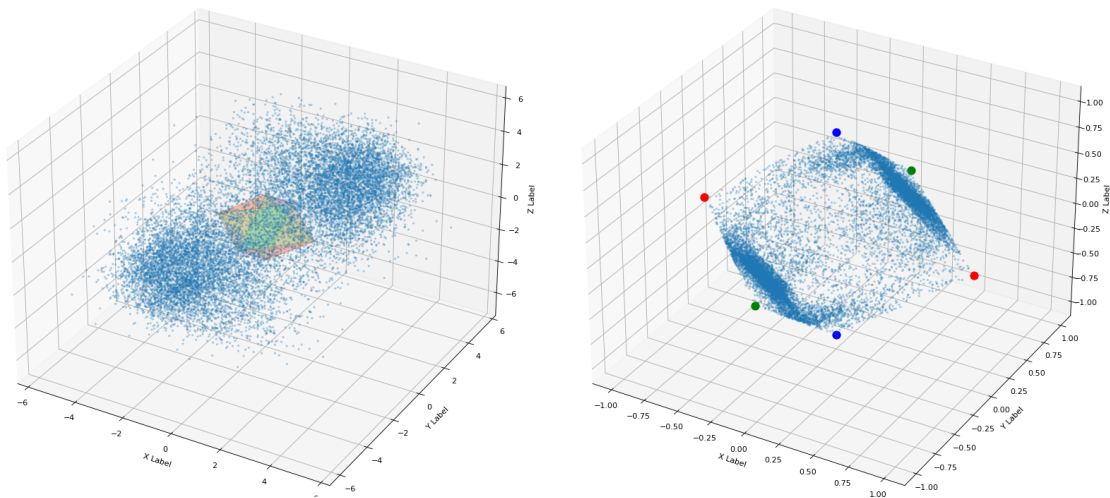


Rysunek 3.7: Pocięta siatka oktaedru. Wszystkie ściany zostały ułożone tak, aby każda krawędź pasowała do najbliższego sąsiada. W lewym górnym rogu pierwszy oktant $(+++)$.

Rozdział 4

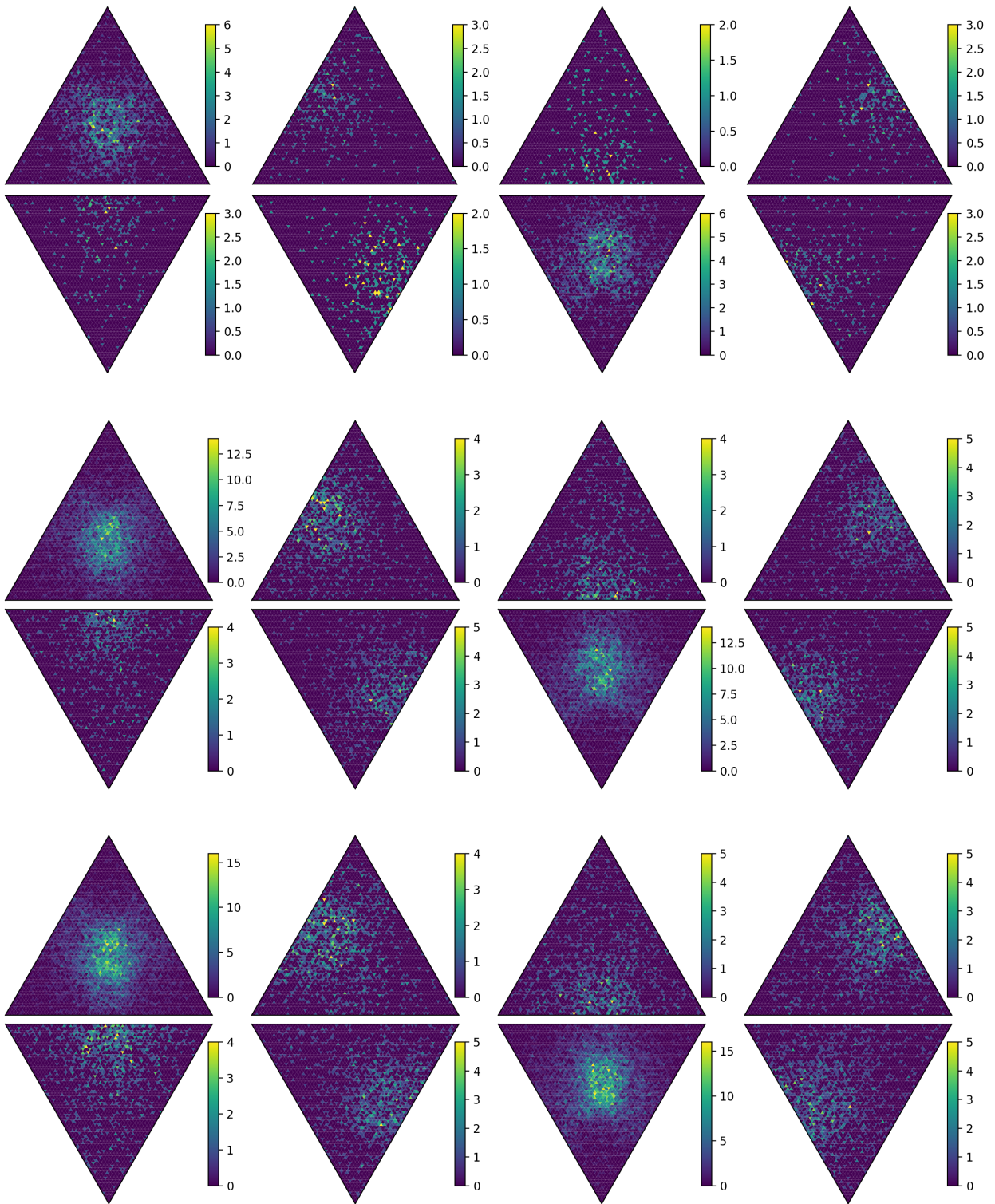
Analiza danych z obliczeń numerycznych

Dzięki uprzejmości Pani profesor A. Emmanouilidou, algorytm mógł zostać przetestowany na danych numerycznych opisanych w pracy [8]. Są to obliczone numerycznie końcowe pędy elektronów pochodzące z procesu niesekwencyjnej potrójnej jonizacji. Jonizacji został poddany atom Neonu za pomocą pojedynczego impulsu laserowego spolaryzowanego liniowo o parametrach: moc szczytowa 1.0-1.6 PW/cm² (10¹⁵W/cm²), czas trwania 25 femtosekund, długości fali 800 nm oraz obwiedni Gaussa. W celu przykładowego porównania przestrzennego rozkładu surowych danych i tych po projekcji na Rys. 4.1 został umieszczony każdy z nich. Na panelu po lewej dane surowe, mało co można z nich odczytać. Na panelu po



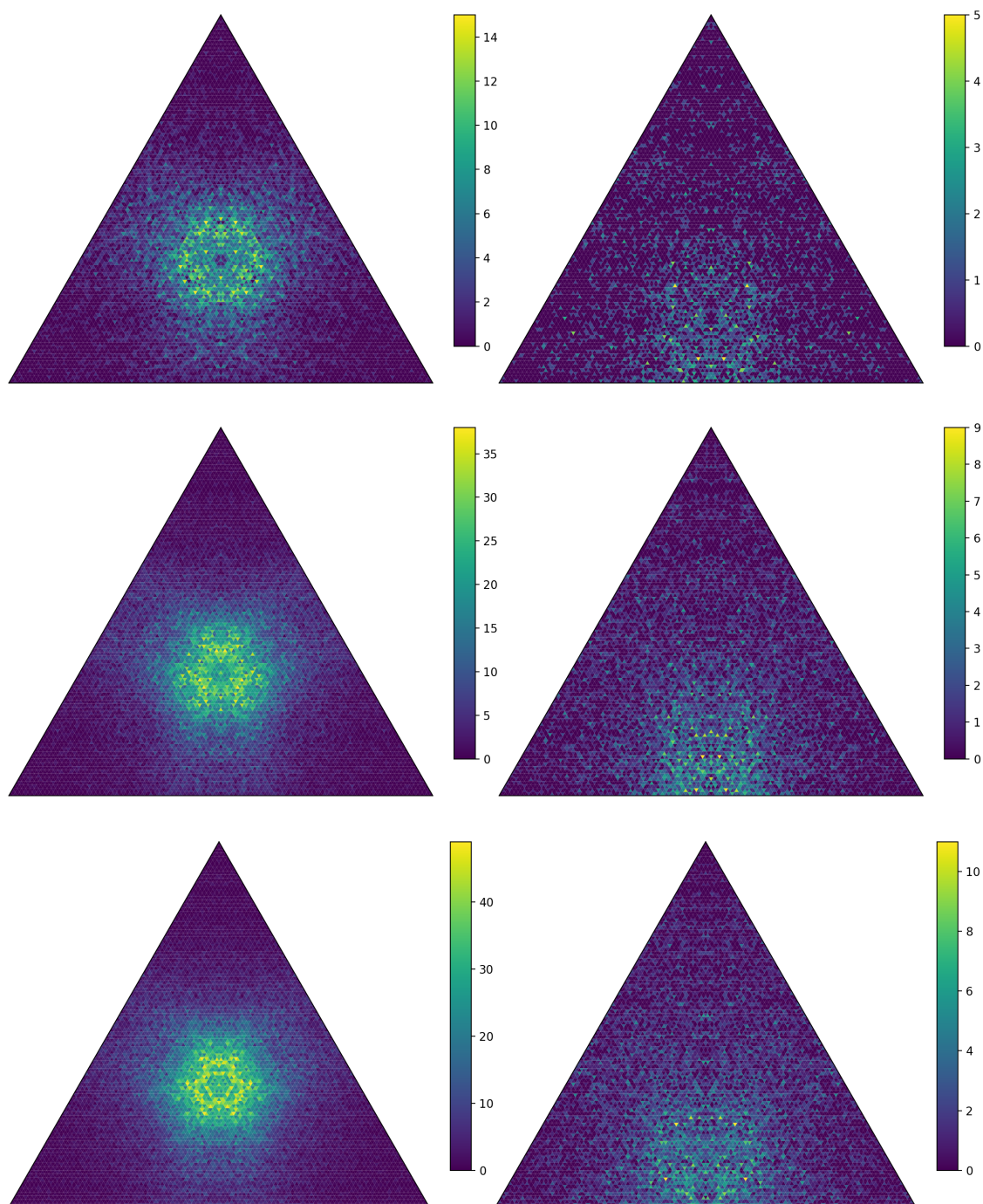
Rysunek 4.1: Po lewej surowe dane. Po prawej obraz danych na oktaedrze. Dane dla mocy lasera 1.3PW/cm²

prawej obraz punktów po rzutowaniu. Ten drugi wykres daje już nieco informacji co do rozmieszczenia punktów, ale bez możliwości interaktywnego obracania figury nie da się odczytać informacji w sposób szczegółowy. Rozwiązując ten problem rozkładamy oktaedr na siatkę, co daje możliwość osobnej analizy każdej ściany nawet na kartce papieru. Ten proces został przeprowadzony już dla wszystkich trzech pakietów danych o różnej mocy lasera i zostały umieszczone na Rys. 4.2.



Rysunek 4.2: Siatka oktaedru. NSTI Neonu kolejno dla mocy lasera: 1.0, 1.3, 1.6 PW/cm^2 . Słupki znajdujące się po prawej stronie każdego z trójkątów przedstawia skale informującą o ilości zliczeń.

Analizując wstępnie rozkłady na siatkach, od razu rzuca się w oczy fakt, że pęd w ćwiartkach $(+++)$ oraz $(---)$ ma raczej tendencje do rozkładania się mniej-więcej po równo dla każdego z elektronów. Są to jedyne sytuacje w których trzy elektrony lecą w tym samym kierunku. Natomiast na wszystkich pozostałych ścianach oktaedru, punkty lokalizują się blisko środka jednej z krawędzi. Taki rozkład sugeruje, że elektron poruszający się w przeciwnym kierunku niż dwa pozostałe unosi ze sobą bardzo niewielką część pędu w stosunku do pozostałych. Dwa pozostałe dzielą się pędem mniej-więcej po równo. Widać również znacznie rosnącą liczbę zdarzeń potrójnej jonizacji wraz ze wzrostem mocy impulsu lasera. Analizowane dane są efektem obliczeń klasycznych, czyli każdy elektron biorący udział w całym procesie jonizacji był oznaczony "etykietą". Z punktu widzenia fizyki kwantowej elektrony są nierozróżnialne. Możemy wykorzystać ten fakt, by sztucznie, ale legalnie zwiększyć statystykę zamieniając ze sobą miejscami nierozróżnialne elektrony. Na ścianach oktaedru można zaobserwować dwie różne sytuacje: albo wszystkie elektrony poruszają się w tym samym kierunku albo dwa elektrony poruszają się w tym samym kierunku, a jeden w przeciwnym. Kierując się takim rozróżnieniem, zostały stworzone dodatkowe dwa wykresy. Dla każdego z przypadków wykres ze wszystkimi punktami, które go spełniają (Rys. 4.3). Wspomniane wcześniej tendencje stają się bardziej wyraźne, tzn. wraz ze wzrostem intensywności impulsu laserowego liczba skorelowanych potrójnych ucieczek rośnie.



Rysunek 4.3: Po lewej pędy elektronów w tym samym kierunku. Po prawej jeden (w górnym rogu) w innym kierunku niż pozostałe. NSTI neonu przy mocy lasera od góry kolejno: 1.0, 1.3, 1.6 PW/cm². Skala przy trójkątach mówi o ilości zliczeń

Rozdział 5

Podsumowanie

Stworzony algorytm może być zastosowany do danych pochodzących z obliczeń numerycznych, co zostało pokazane w poprzednim rozdziale. Jednak pozwala on również na przekształcania danych z pomiarów eksperymentalnych. Jedyna różnica będzie polegać na tym, że od razu będą dane liczby zliczeń zdarzeń o zadanych współrzędnych pędowych. W momencie tworzenia histogramu trójkątnego trzeba będzie je uwzględnić. Przy delikatnej modyfikacji, algorytm powinien się sprawdzić również przy przetwarzaniu danych z obliczeń kwantowych. W trakcie pisania pracy zostały poczynione takie próby. Po drodze pojawiły się jednak problemy związane z ograniczeniami sprzętowymi, a dokładnie ograniczenia ilości pamięci RAM. W trakcie pracy korzystałem ze standardowego na te czasy komputera osobistego, który dysponował pamięcią RAM o pojemności 16GB. Wynikowy plik obliczeń kwantowych, zajmował podobną ilość i z tego względu nie udało się sfinalizować tego przekształcenia. Istnieje jednak możliwość zmiany sposobu przetwarzania danych lub skorzystanie z mocniejszej maszyny, aby takich przekształceń dokonać.

Dodatek A

Skrypt

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pandas import read_csv
4 import mpltern
5
6
7 #file names
8 f1 = 'final_momenta_1.0PW.txt'
9 f2 = 'final_momenta_1.3PW.txt'
10 f3 = 'final_momenta_1.6PW.txt'
11
12 #file from which data will be extracted
13 file = f3
14
15
16 #The code in this location needs to be adapted depending on the data
    format
17
18 #Reading data
19 #the data does not contain a column header
20 #data is saved as: pnx, pny, pnz, p1x, p1y, p1z, p2x, p2y, p2z, p3x,
    p3y, p3z
21 #x:0, y:1, z:2 - (z is axis of polarization)
22 axis = 2
23 dataframe = read_csv(file, decimal = '.', dtype = float, sep=' ', header=
    None, usecols=[3+axis,6+axis,9+axis])
24
25 #p1z --> x1, p2z --> y1, p3z --> z1
26 x1 = np.array(dataframe.values[:,0])
27 y1 = np.array(dataframe.values[:,1])
28 z1 = np.array(dataframe.values[:,2])
29
30 #making space in memory
31 del dataframe
32
33
34 #Add distance from point to side(axis yz, zx, xy)
35 def add_to(oktant,distance):
```

```

36     oktant[0].append(distance[0])
37     oktant[1].append(distance[1])
38     oktant[2].append(distance[2])
39
40 #Add distance from point to side(axis yz, zx, xy)
41 # symmetrization - electrons are indistinguishable
42 def add_with_all_permutations(oktant,distance):
43     oktant[0].append(distance[0])
44     oktant[1].append(distance[1])
45     oktant[2].append(distance[2])
46
47     oktant[1].append(distance[0])
48     oktant[2].append(distance[1])
49     oktant[0].append(distance[2])
50
51     oktant[2].append(distance[0])
52     oktant[1].append(distance[1])
53     oktant[0].append(distance[2])
54
55     oktant[0].append(distance[0])
56     oktant[2].append(distance[1])
57     oktant[1].append(distance[2])
58
59     oktant[1].append(distance[0])
60     oktant[0].append(distance[1])
61     oktant[2].append(distance[2])
62
63     oktant[2].append(distance[0])
64     oktant[0].append(distance[1])
65     oktant[1].append(distance[2])
66
67 #Add distance from point to side(axis yz, zx, xy)
68 # symmetrization - electrons which are moving in the same direction are
    indistinguishable
69 #The first value in array 'side' have different direction
70 #different - is a number of different value in distance
71 def add_with_two_permutations(oktant, distance, different):
72     oktant[0].append(distance[different])
73     oktant[1].append(distance[(different+1)%3])
74     oktant[2].append(distance[(different+2)%3])
75
76     oktant[0].append(distance[different])
77     oktant[1].append(distance[(different+2)%3])
78     oktant[2].append(distance[(different+1)%3])
79
80
81 #preparing arrays
82
83 #temporary distance of [yz, zx, xy]
84 r = [0,0,0]
85
86 #Arrays representing faces of octahedron in following order:
87 #(+++),(-++),(--+),(+--+),(++-),(-+-),(--),(+--).
88 p0 = [[],[],[ ]]

```

```

89 p1 = [[],[],[]]
90 p2 = [[],[],[]]
91 p3 = [[],[],[]]
92 p4 = [[],[],[]]
93 p5 = [[],[],[]]
94 p6 = [[],[],[]]
95 p7 = [[],[],[]]
96
97 #array for the same momentum direction (+++ and ---)
98 main = [[],[],[]]
99
100 #array where one momentum is in the opposite direction to the others
101 side = [[],[],[]]
102
103 for i in range(len(x1)):
104
105     #absolute value of the vector component
106     dx=abs(x1[i])
107     #absolute value of the vector component
108     dy=abs(y1[i])
109     #absolute value of the vector component
110     dz=abs(z1[i])
111
112     #plane equation
113     t = dx+dy+dz
114
115     #normalized distance from point (x,y,z) to yz line
116     r[0] = dx/t
117     #normalized distance from point (x,y,z) to zx line
118     r[1] = dy/t
119     #normalized distance from point (x,y,z) to xy line
120     r[2] = dz/t
121
122     #adding distances to each octant
123     if (x1[i]>=0 and y1[i]>=0 and z1[i]>=0):
124         add_to(p0,r)
125         add_with_all_permutations(main,r)
126
127     elif (x1[i]<0 and y1[i]>=0 and z1[i]>=0):
128         add_to(p1,r)
129         add_with_two_permutations(side,r,0)
130
131     elif (x1[i]<0 and y1[i]<0 and z1[i]>=0):
132         add_to(p2,r)
133         add_with_two_permutations(side,r,2)
134
135     elif (x1[i]>=0 and y1[i]<0 and z1[i]>=0):
136         add_to(p3,r)
137         add_with_two_permutations(side,r,1)
138
139     #####
140     elif (x1[i]>=0 and y1[i]>=0 and z1[i]<0):
141         add_to(p4,r)
142         add_with_two_permutations(side,r,2)

```

```

143
144     elif (x1[i]<0 and y1[i]>=0 and z1[i]<0):
145         add_to(p5,r)
146         add_with_two_permutations(side,r,1)
147
148     elif (x1[i]<0 and y1[i]<0 and z1[i]<0):
149         add_to(p6,r)
150         add_with_all_permutations(main,r)
151
152     elif (x1[i]>=0 and y1[i]<0 and z1[i]<0):
153         add_to(p7,r)
154         add_with_two_permutations(side,r,1)
155
156     #normalization check
157     #print(r[0],",",r[1],",",r[2]," : ",(r[0]+r[1]+r[2]))
158
159 #making space in memory
160 del x1
161 del y1
162 del z1
163
164
165
166 #Print the octahedron as a grid where the closest sides match up
167
168 #group all octants (to easier print)
169 p = [p0,p1,p2,p3,p4,p5,p6,p7]
170
171 #All labels (to easier print)
172 #labelsP1 = ['+P1','-P1']
173 #labelsP2 = ['+P2','+P3','-P2','-P3', '+P3','-P2','-P3','+P2']
174 #labelsP3 = ['+P3','-P2','-P3','+P2', '+P2','+P3','-P2','-P3']
175
176 #Just single (they don't overlap)
177 labelsP1 = ['+P1','-P1']
178 labelsP2 = ['+P2','','','','', '','','','']
179 labelsP3 = ['+P3','-P2','-P3','+P2', '','','','','']
180
181 #print helpers
182 rotation = 0
183 down = 0
184
185 #the number of triangles into which the side of the triangle will be
    divided
186 grid = 60
187
188 #creating a figure
189 #increase "dpi" to improve detail quality
190 fig = plt.figure(figsize=(12,5), dpi=200)
191 fig.subplots_adjust(
192     left=0.1,
193     right=0.9,
194     bottom=0.1,
195     top=1,

```

```

196     wspace=0.5,
197     hspace=0.5,
198 )
199
200 for i,ps in enumerate(p):
201     if(i==4):
202         rotation = 180
203         down = 1
204
205     ax = fig.add_subplot(2, 4, i+1, projection='ternary', rotation=
rotation)
206     ax.set_tlabel(labelsP1[down])
207     ax.set_llabel(labelsP2[i])
208     ax.set_rlabel(labelsP3[i])
209
210     #disable ticks on axis
211     #comment 3 lines below to restore default settings
212     ax.taxis.set_ticks([])
213     ax.raxis.set_ticks([])
214     ax.laxis.set_ticks([])
215
216     #set the axis labels horizontally
217     #comment 3 lines below to restore default settings
218     ax.taxis.set_label_rotation_mode('horizontal')
219     ax.laxis.set_label_rotation_mode('horizontal')
220     ax.raxis.set_label_rotation_mode('horizontal')
221
222     #Here we put the data of each oktant as (ps[0], ps[1], ps[2])
223     #You can use line below (hexbin) instead of (tribin)
224     #pc = ax.hexbin(ps[2], ps[(i+down)%2], ps[(i+1+down)%2], gridszize =
grid, edgecolors=None)
225     pc = ax.tribin(ps[2], ps[(i+down)%2], ps[(i+1+down)%2], gridszize =
grid, edgecolors=None)
226
227     cax = ax.inset_axes([0.98, 0.1, 0.05, 0.8])
228     colorbar = fig.colorbar(pc, cax=cax)
229     colorbar.set_label("Count", rotation=270, va="baseline")
230
231
232 plt.tight_layout()
233
234 # file[-9:-4] --> this fragment contains the power of the laser used to
obtain the data
235 plt.savefig('oktants '+file[-9:-4]+'.png')
236 plt.show()
237
238 #draw two triangles (electrons with the same direction and electrons with
the different directions)
239
240 #creating a figure
241 #increase "dpi" to improve detail quality
242 fig = plt.figure(figsize=(12,6), dpi=200)
243 fig.subplots_adjust(
244     left=0.1,

```

```

245     right=0.9,
246     bottom=0.1,
247     top=0.9,
248     wspace=0.5,
249     hspace=0.5,
250 )
251
252 rotation = 0
253 down = 0
254
255 #the number of triangles into which the side of the triangle will be
    divided
256 grid = 100
257
258 ax = fig.add_subplot(1, 2, 1, projection='ternary', rotation=rotation)
259 #ax.set_title('all electrons one direction')
260
261 #Printing the data (ps[0], ps[1], ps[2])
262 #pc = ax.hexbin(main[0], main[1], main[2], gridsize = grid, edgecolors=
    None)
263 pc = ax.tribin(main[0], main[1], main[2], gridsize = grid, edgecolors=None
    )
264 cax = ax.inset_axes([1.05, 0.1, 0.05, 0.9])
265 colorbar = fig.colorbar(pc, cax=cax)
266 colorbar.set_label("Count", rotation=270, va="baseline")
267
268 #disable ticks on axis
269 #comment 3 lines below to restore default settings
270 ax.taxis.set_ticks([])
271 ax.raxis.set_ticks([])
272 ax.laxis.set_ticks([])
273
274 ax = fig.add_subplot(1, 2, 2, projection='ternary', rotation=rotation)
275 #ax.set_title('one electron in opposite direction than others')
276
277 #Printing the data (ps[0], ps[1], ps[2])
278 #pc = ax.hexbin(side[0], side[1], side[2], gridsize = grid, edgecolors=
    None)
279 pc = ax.tribin(side[0], side[1], side[2], gridsize = grid, edgecolors=None
    )
280 cax = ax.inset_axes([1.05, 0.1, 0.05, 0.9])
281 colorbar = fig.colorbar(pc, cax=cax)
282 colorbar.set_label("Count", rotation=270, va="baseline")
283
284 #disable ticks on axis
285 #comment 3 lines below to restore default settings
286 ax.taxis.set_ticks([])
287 ax.raxis.set_ticks([])
288 ax.laxis.set_ticks([])
289
290 plt.tight_layout()
291 plt.savefig('main info '+file[-9:-4]+'.png')
292 plt.show()

```

Bibliografia

- [1] J. H. Thiede, “Frequency dependence of double ionization in strong laser fields,” Master’s thesis, Department of Physics, Philipps University in Marburg, 2011.
- [2] B. T. and F. Krausz, “Intense few-cycle laser fields: Frontiers of nonlinear optics,” *Rev. Mod. Phys.*, vol. 72, p. 545, 2000.
- [3] A. l’Huillier, L. Lompre, G. Mainfray, and C. Manus, “Multiply charged ions induced by multiphoton absorption in rare gases at $0.53\ \mu\text{m}$,” *Phys. Rev. A*, vol. 27, no. 5, p. 2503, 1983.
- [4] A. l’Huillier, L. A. Lompre, G. Mainfray, and C. Manus, “Multiply charged ions formed by multiphoton absorption processes in the continuum,” *Phys. Rev. Lett.*, vol. 48, pp. 1814–1817, Jun 1982.
- [5] P. B. Corkum, “Plasma perspective on strong field multiphoton ionization,” *Phys. Rev. Lett.*, vol. 71, p. 1994, 1993.
- [6] P. B. Corkum and F. Krausz, “Attosecond science,” *Nat. Phys.*, vol. 3, p. 381, 2007.
- [7] A. Staudte, C. Ruiz, M. Schöffler, S. Schössler, D. Zeidler, T. Weber, M. Meckel, D. Villeneuve, P. Corkum, A. Becker *et al.*, “Binary and recoil collisions in strong field double ionization of helium,” *Phys. Rev. Lett.*, vol. 99, p. 263002, 2007.
- [8] A. Emmanouilidou, M. B. Peters, and G. P. Katsoulis, “Singularity in the electron-core potential as a gateway to accurate multielectron ionization spectra in strongly driven atoms,” *Phys. Rev. A*, vol. 107, p. L041101, 2023.
- [9] D. K. Efimov, A. Maksymov, M. Ciappina, J. S. Prauzner-Bechcicki, M. Lewenstein, and J. Zakrzewski, “Three-electron correlations in strong laser field ionization,” *Optics Express*, vol. 29, p. 26526, August 2021.
- [10] Y. Ikeda, “yuzie007/mpltern: 1.0.2,” aug 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.8289090>
- [11] W. R. Inc., “Mathematica, Version 13.3,” champaign, IL, 2023. [Online]. Available: <https://www.wolfram.com/mathematica>